

A. I used CLAUDE and COPILOT.

B. I accessed CLAUDE through the app on my computer. I accessed COPILOT on the website through the Microsoft Edge browser on my computer.

C. Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

D. **CLAUDE:**

```
#include <stdio.h>
```

```
int main(void) {  
    int secret = 7;  
    int guess;  
    int attempts = 3;
```

```
    printf("Guess a number between 1 and 10.\n");
```

```
    for (int i = 1; i <= attempts; i++) {  
        printf("Attempt %d/%d. Enter your guess: ", i, attempts);  
        scanf("%d", &guess);
```

```
        if (guess == secret) {
```

```

        printf("Correct! You win!\n");
        break;
    } else if (guess < secret) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }

    if (i == attempts && guess != secret) {
        printf("You lose! The number was %d.\n", secret);
    }
}

return 0;
}

```

COPILLOT:

```

#include <stdio.h>

int main() {
    int secret = 7;        // fixed secret number
    int guess;
    int attempts = 3;

    printf("Guess a number between 1 and 10.\n");

    for (int i = 1; i <= attempts; i++) {
        printf("Attempt %d/%3d. Enter your guess: ", i, attempts);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            return 0; // stop immediately
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    printf("Sorry, you lose. The number was %d.\n", secret);
    return 0;
}

```

- E. Although the two codes are very similar, there are a few distinct differences.

EXECUTION TIME: First, the COPILOT version returns 0 instead of a break whenever the user guesses the number. This allows for one less if-statement compared to the CLAUDE version. The break will only exit out of the if-else statement block, while the return 0 completes the main function in its entirety. This means that if the user guesses the number correctly the second half of the code won't need to run, giving COPILOT a shorter execution time. They also both use for-loops, which have better execution time than while-loops.

MAINTAINABILITY: Additionally, both programs need some more comments to improve maintainability. If you are unfamiliar with the program, being provided with comments can help you maintain and change the code. However, the COPILOT version also wins in this regard as it has a few comments, compared to none.

CORRECTNESS: As far as correctness, both codes function properly. The one issue is that they do not error trap for non-number inputs. The COPILOT code also adds in an extra tab when printing which guess the user is on out of three, throwing off the expected formatting. After compiling, this causes the program to end without accepting any more inputs, decreasing correctness.

SPACE COMPLEXITY: Finally, since they include the same finite number of variables, the space complexity and amount of memory used will be the same. Since there is no use of recursion or any loops that create more variables, the space complexity is very small.

- F. I decided to use the COPILOT version as it excels in maintainability and execution time.
- G. The first change I will make is adding more comments to increase maintainability. I will also fix the format error and check for edge cases in order to fix the correctness. I will do this by removing the 3 from “%d/%d”, similar to how CLAUDE had done it. I will also repeatedly ask the user for input until it is valid. While this will increase the execution time, I believe that the increased correctness is worth it for the little increase in time complexity. I also used COPILOT in order to implement this change.